ReceiptRadar — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── samples/
│   ├── approved.txt
│   ├── banned_category.txt
│   └── over_limit.txt
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── loader.py
│   ├── pipeline.py
│   ├── policy.py
│   └── schemas.py
├── tests/
│   ├── integration/
│   │   └── test_live_extract.py
│   └── unit/
│       ├── test_config.py
│       └── test_policy.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

Setup files

`requirements.txt`:

```
langchain-openai
langchain-community
openai
pydantic
python-dotenv
```

(Add `pypdf` only if you load `.pdf` receipts.)

`.env.example`:

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`:

```
.env
.venv/
__pycache__/
*.pyc
```

`LICENSE` is the standard MIT text. Every folder under `src/` gets an empty `__init__.py`.

Sample receipts (`samples/`)

```
# samples/approved.txt
Merchant: Corner Cafe
Amount: 12.00
Category: meals
Note: team lunch
```

```
# samples/over_limit.txt
Merchant: Grand Hotel
Amount: 200.00
Category: lodging
```

```
# samples/banned_category.txt
Merchant: The Pub
```

```
Amount: 30.00
Category: alcohol
```

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass


class UnsupportedFileError(Exception):
    """The receipt file type is not supported."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"


def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

The shape, the loader, the policy

```
# src/schemas.py
# The typed shape of a receipt. The model fills this in; the policy validator
# checks it. Fields are optional so a missing one becomes a problem, not an
# invented value. has_note defaults to False (no note attached).
from typing import Optional

from pydantic import BaseModel


class Receipt(BaseModel):
```

```
    merchant: Optional[str] = None
    amount: Optional[float] = None
    category: Optional[str] = None
    has_note: bool = False
```

```
# src/loader.py
# Read a receipt file into plain text. .txt/.md are read directly; .pdf uses a
# document loader (Doc08). The model only ever sees this text.
from src.exceptions import UnsupportedFileError


def load_receipt_text(path: str) -> str:
    lower = path.lower()
    if lower.endswith((".txt", ".md")):
        with open(path) as receipt_file:
            return receipt_file.read()
    if lower.endswith(".pdf"):
        # needs: pip install pypdf
        from langchain_community.document_loaders import PyPDFLoader
        pages = PyPDFLoader(path).load()
        texts = []
        for page in pages:
            texts.append(page.page_content)
        return "\n".join(texts)
    raise UnsupportedFileError(f"Unsupported receipt file type: {path}")
```

```
# src/policy.py
# The company expense policy as deterministic code, so the same receipt always
# gets the same decision. Testable with hand-made Receipt objects - no model.
from src.schemas import Receipt

NOTE_THRESHOLD = 75.0
BANNED_CATEGORIES = ("alcohol", "gift", "gifts")


def check_policy(receipt: Receipt) -> list:
    problems = []
    if not receipt.merchant:
        problems.append("missing merchant")
    if receipt.amount is None:
        problems.append("missing amount")
    else:
        if receipt.amount > NOTE_THRESHOLD and not receipt.has_note:
            problems.append(
                f"amount {receipt.amount} is over the {NOTE_THRESHOLD} "
```

```
                f"note threshold and has no note"
            )
    if receipt.category is not None:
        if receipt.category.lower() in BANNED_CATEGORIES:
            problems.append(
                f"category '{receipt.category}' is not reimbursable"
            )
    return problems
```

The pipeline

```
# src/pipeline.py
# Tie it together: load the file, extract a typed Receipt with the model, run
# the deterministic policy check, return the data plus any policy problems.
from dataclasses import dataclass

from langchain_core.prompts import ChatPromptTemplate

from src.loader import load_receipt_text
from src.policy import check_policy
from src.schemas import Receipt
from src.utils.logger import get_logger

EXTRACT_PROMPT = (
    "Extract the receipt fields from the text below. Set has_note to true "
    "only if a note/justification is present. If a field is missing, leave "
    "it empty - never invent a value.\n\nReceipt text:\n{text}"
)


@dataclass
class Result:
    receipt: Receipt
    problems: list


def extract_receipt(model, text: str) -> Receipt:
    prompt = ChatPromptTemplate.from_template(EXTRACT_PROMPT)
    chain = prompt | model.with_structured_output(Receipt)
    return chain.invoke({"text": text})


def process_receipt(model, path: str) -> Result:
    logger = get_logger(__name__)
    text = load_receipt_text(path)
```

```
    receipt = extract_receipt(model, text)
    problems = check_policy(receipt)
    logger.info(f"processed {path}: {len(problems)} policy problem(s)")
    return Result(receipt=receipt, problems=problems)
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import process_receipt


def main():
    if len(sys.argv) < 2:
        print("usage: python main.py <receipt-file>")
        return
    path = sys.argv[1]
    config = load_config()
    model = get_model(config)
    result = process_receipt(model, path)
    receipt = result.receipt

    print(f"\nReceiptRadar result for: {path}")
    print(f"  merchant: {receipt.merchant}")
    print(f"  amount:   {receipt.amount}")
    print(f"  category: {receipt.category}")

    if not result.problems:
        print("\nApproved: no policy problems.")
    else:
        print("\nPolicy problems:")
        for problem in result.problems:
            print(f"  - {problem}")


if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "RECEIPTRADAR_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

The policy test is the important one — it proves the rules with **no model and no key**, using hand-made `Receipt` objects:

```
# tests/unit/test_policy.py
# Run from the repo root: python -m tests.unit.test_policy
from src.policy import check_policy
from src.schemas import Receipt


def has_problem(problems, text: str) -> bool:
    for problem in problems:
        if text in problem:
            return True
```

```
    return False


def test_small_receipt_is_approved() -> None:
    receipt = Receipt(merchant="Cafe", amount=12.0, category="meals")
    result = "OK"
    if check_policy(receipt) != []:
        result = "WRONG"
    print(f"case 1 (small receipt -> approved)   -> {result}")


def test_over_limit_no_note_is_flagged() -> None:
    receipt = Receipt(merchant="Hotel", amount=200.0, category="lodging")
    result = "OK"
    if not has_problem(check_policy(receipt), "note threshold"):
        result = "WRONG"
    print(f"case 2 (over limit, no note -> flag) -> {result}")


def test_over_limit_with_note_is_ok() -> None:
    receipt = Receipt(merchant="Hotel", amount=200.0, category="lodging",
                      has_note=True)
    result = "OK"
    if has_problem(check_policy(receipt), "note threshold"):
        result = "WRONG"
    print(f"case 3 (over limit, with note -> ok) -> {result}")


def test_banned_category_is_flagged() -> None:
    receipt = Receipt(merchant="Pub", amount=30.0, category="Alcohol")
    result = "OK"
    if not has_problem(check_policy(receipt), "not reimbursable"):
        result = "WRONG"
    print(f"case 4 (banned category -> flagged)  -> {result}")


if __name__ == "__main__":
    test_small_receipt_is_approved()
    test_over_limit_no_note_is_flagged()
    test_over_limit_with_note_is_ok()
    test_banned_category_is_flagged()
```

```
# tests/integration/test_live_extract.py
# Real model. Run from the repo root:
#   python -m tests.integration.test_live_extract
from src.config import load_config
```

```
from src.models.llm import get_model
from src.pipeline import process_receipt


def test_approved_receipt_clears() -> None:
    config = load_config()
    model = get_model(config)
    result = process_receipt(model, "samples/approved.txt")
    status = "OK"
    if result.problems:
        status = "WRONG"
    print(f"case 1 (approved receipt clears) -> {status}")


def test_over_limit_is_flagged() -> None:
    config = load_config()
    model = get_model(config)
    result = process_receipt(model, "samples/over_limit.txt")
    flagged = False
    for problem in result.problems:
        if "note threshold" in problem:
            flagged = True
    status = "OK"
    if not flagged:
        status = "WRONG"
    print(f"case 2 (over limit is flagged)   -> {status}")


if __name__ == "__main__":
    test_approved_receipt_clears()
    test_over_limit_is_flagged()
```

Expected unit output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (small receipt -> approved)   -> OK
case 2 (over limit, no note -> flag) -> OK
case 3 (over limit, with note -> ok) -> OK
case 4 (banned category -> flagged)  -> OK
```

The whole project, in one sentence

A document loader reads a receipt into text, `with_structured_output` extracts it into a typed `Receipt`, and a deterministic policy validator enforces the expense rules — over-limit-no-note and banned categories always caught — so the model reads and your code decides what's reimbursable.